

第四讲：特征检测



授课人：王国泰

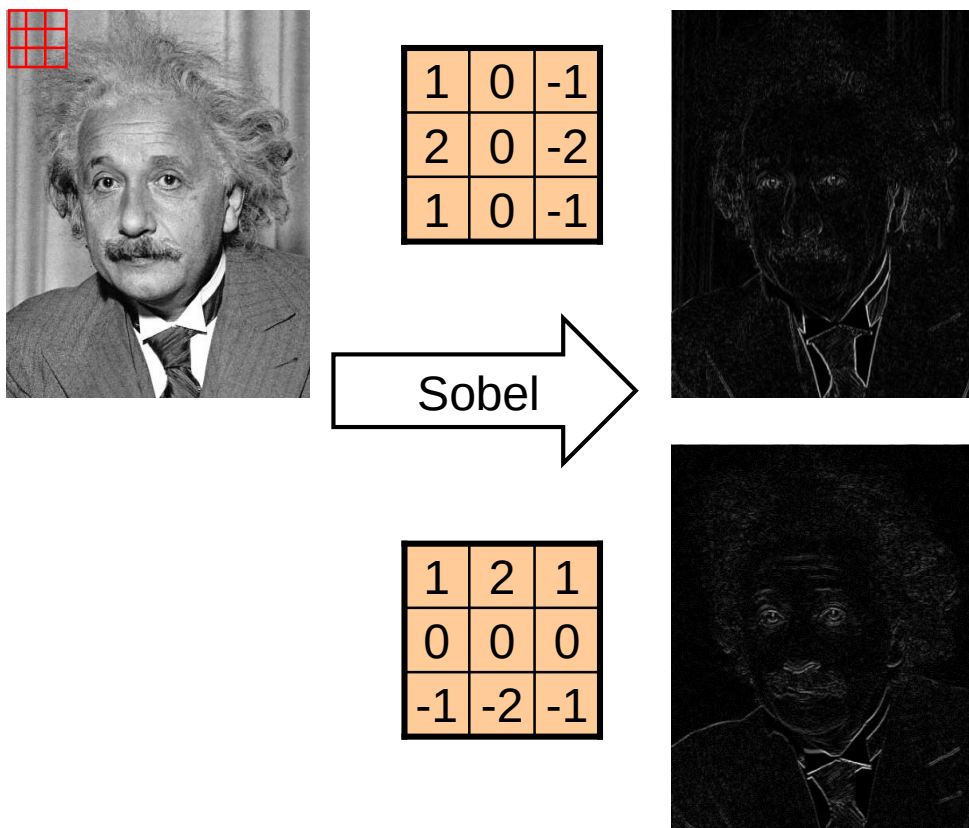
电子科技大学机械与电气工程学院

2026-3-12

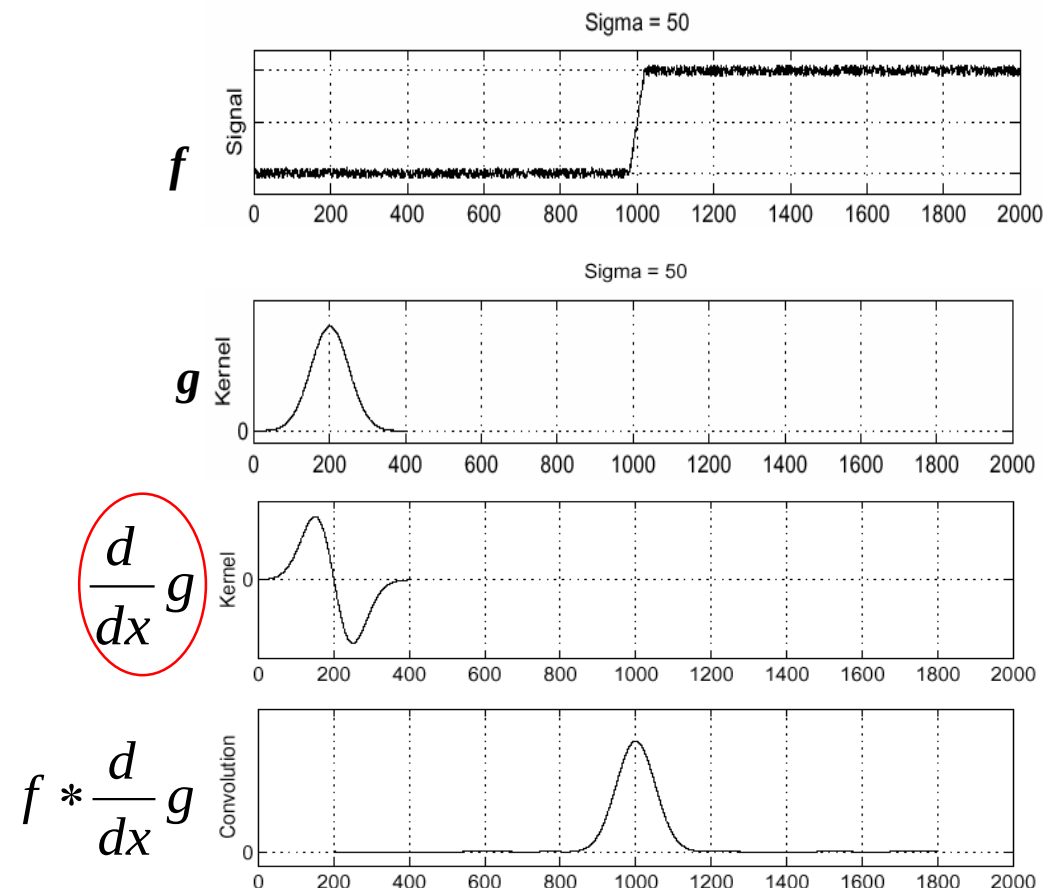


知识回顾：图像滤波与边缘检测

- Sobel算子



- 高斯差分滤波器



Today: Feature Detection 特征检测



- Example: Panorama stitching 全景拼接
 - We have two images – how do we combine them?



Today: Feature Detection 特征检测

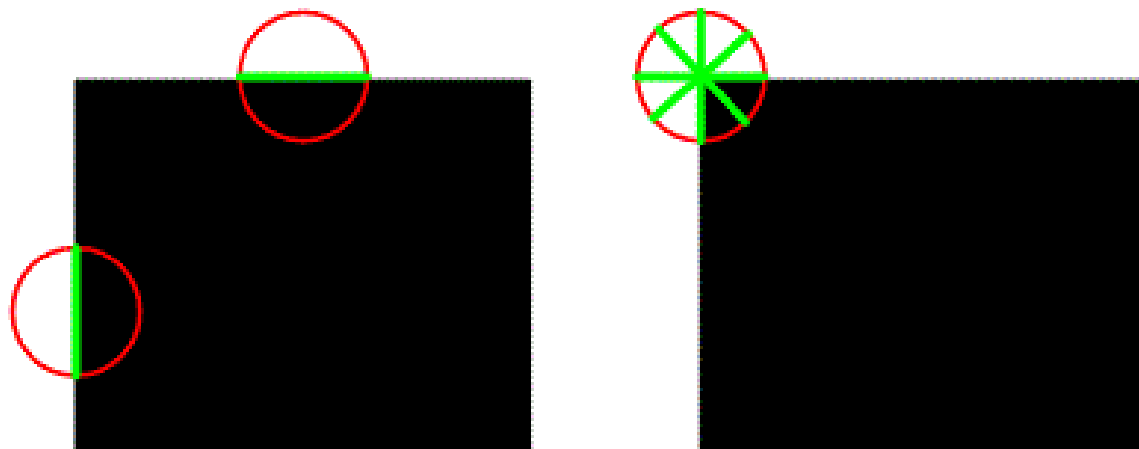
- Example: 3D reconstruction



Outline

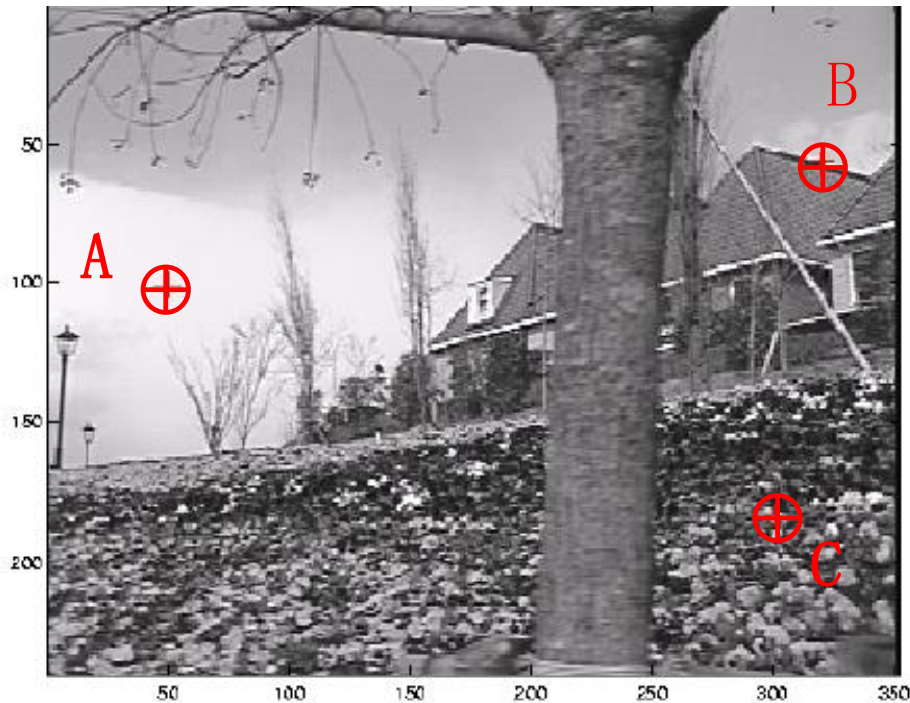


- 1, Corner Detection 角点检测
- 2, Edge Detection 边缘检测
- 3, Line Detection 直线检测



Corner Detection by Auto-correlation

- Change in appearance of window $w(x,y)$ for shift $[u,v]$:



自相关函数: 对于给定的偏移量 (u,v) ，计算像素 (x,y) 和附近像素 $(x+u, y+v)$ 的差值的平方，并在所有像素上加权求和

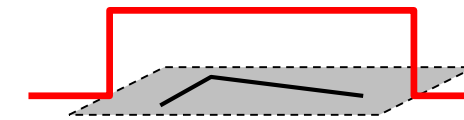
$$E(u,v) = \sum_{x,y} w(x,y) [I(x+u, y+v) - I(x,y)]^2$$

Window
function

Shifted
intensity

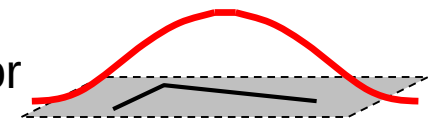
Intensity

$w(x,y) =$



1 in window, 0 outside

or

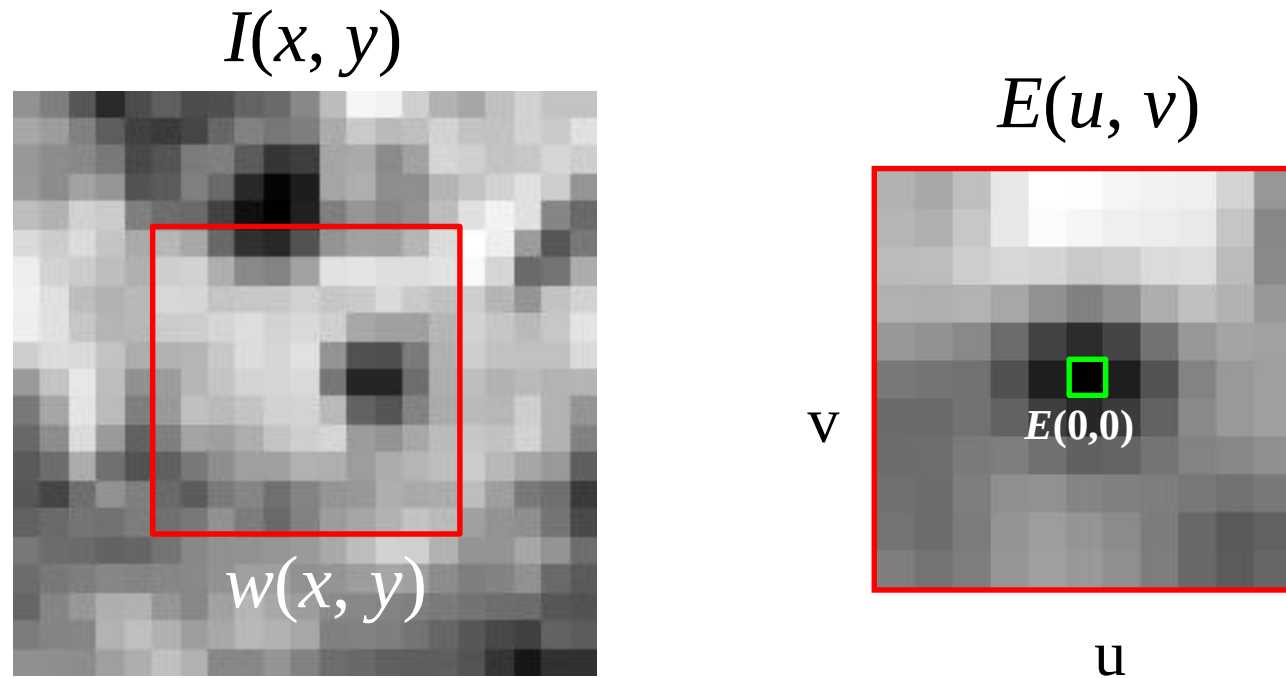


Gaussian

Corner Detection by Auto-correlation

- Change in appearance of window $w(x,y)$ for shift $[u,v]$:

$$E(u, v) = \sum_{x,y} w(x, y) [I(x+u, y+v) - I(x, y)]^2$$



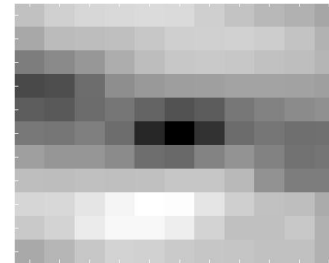
Corner Detection by Auto-correlation

- Auto-correlation surfaces
 - Strong feature points are local minima in $E(u, v)$

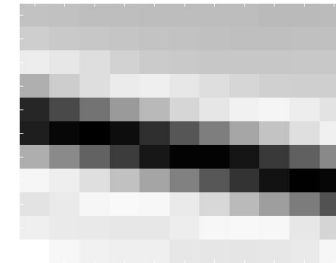
$$E(u, v) = \sum_{x, y} w(x, y) [I(x+u, y+v) - I(x, y)]^2$$



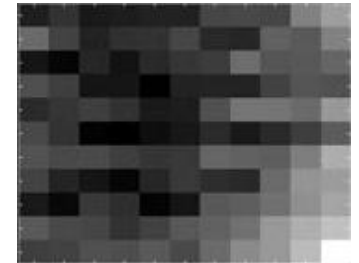
$E(u, v)$ as an image



C

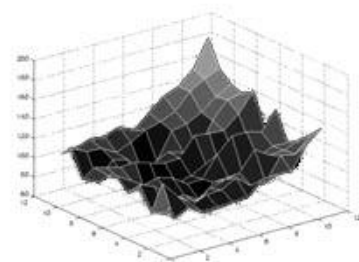
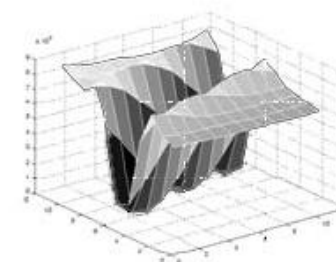
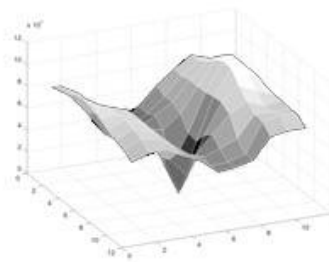


B



A

$E(u, v)$ as a surface

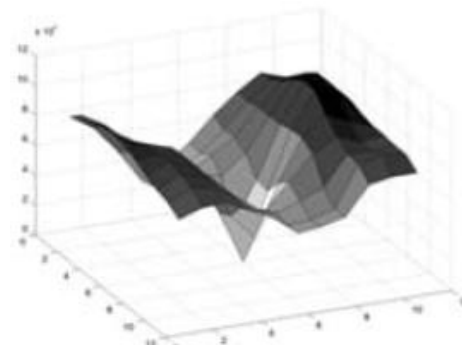
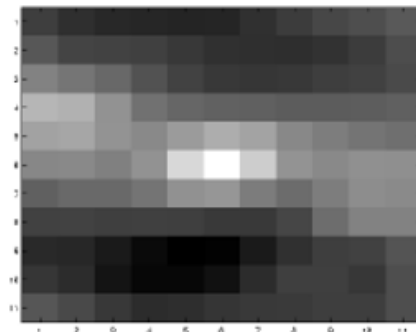


Corner Detection by Auto-correlation

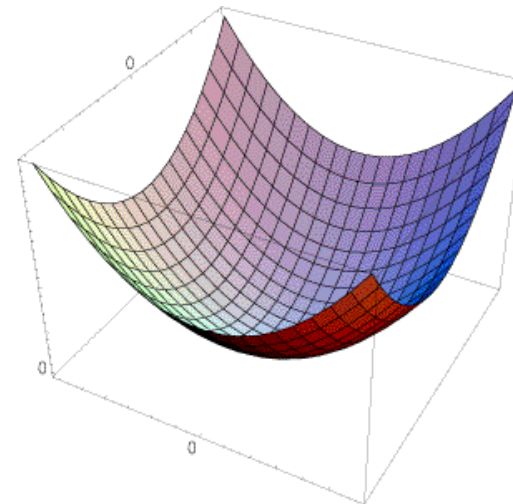
- How to compute auto-correlation efficiently?
 - But this is very **slow to compute naively**.
 - $O(\text{window_width}^2 * \text{shift_range}^2 * \text{image_width}^2)$

$$E(u, v) = \sum_{x, y} w(x, y) [I(x+u, y+v) - I(x, y)]^2$$

- Can we just approximate $E(u, v)$ locally by a quadratic surface?



$\approx$



Taylor expansion 泰勒展开式



- Taylor series expansion

- A function f can be represented by an infinite series of its derivatives at a single point a :

$$f(a) + \frac{f'(a)}{1!}(x-a) + \frac{f''(a)}{2!}(x-a)^2 + \frac{f'''(a)}{3!}(x-a)^3 + \dots$$

- Local quadratic approximation of $E(u,v)$ in the neighborhood of $(0,0)$ is given by the *second-order Taylor expansion*:

$$E(u,v) \approx E(0,0) + [u \ v] \begin{bmatrix} E_u(0,0) \\ E_v(0,0) \end{bmatrix} + \frac{1}{2} [u \ v] \begin{bmatrix} E_{uu}(0,0) & E_{uv}(0,0) \\ E_{uv}(0,0) & E_{vv}(0,0) \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix}$$

↑
Ignore function
value; set to 0

↑
Ignore first derivative,
set to 0

↑
Just look at shape of
second derivative

Corner Detection by Auto-correlation



- Second-order Taylor expansion of $E(u,v)$ about $(0,0)$:

$$E(u,v) = \sum_{x,y} w(x,y) [I(x+u, y+v) - I(x,y)]^2$$

$$E(u,v) \approx E(0,0) + [u \ v] \begin{bmatrix} E_u(0,0) \\ E_v(0,0) \end{bmatrix} + \frac{1}{2} [u \ v] \begin{bmatrix} E_{uu}(0,0) & E_{uv}(0,0) \\ E_{uv}(0,0) & E_{vv}(0,0) \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix}$$

$$E_u(u,v) = \sum_{x,y} 2w(x,y) [I(x+u, y+v) - I(x,y)] I_x(x+u, y+v)$$

$$E_{uu}(u,v) = \sum_{x,y} 2w(x,y) I_x(x+u, y+v) I_x(x+u, y+v) \\ + \sum_{x,y} 2w(x,y) [I(x+u, y+v) - I(x,y)] I_{xx}(x+u, y+v)$$

Corner Detection by Auto-correlation



- Second-order Taylor expansion of $E(u,v)$ about $(0,0)$:

$$E(u,v) = \sum_{x,y} w(x,y) [I(x+u, y+v) - I(x,y)]^2$$

$$E(u,v) \approx E(0,0) + [u \ v] \begin{bmatrix} E_u(0,0) \\ E_v(0,0) \end{bmatrix} + \frac{1}{2} [u \ v] \begin{bmatrix} E_{uu}(0,0) & E_{uv}(0,0) \\ E_{uv}(0,0) & E_{vv}(0,0) \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix}$$

$$E_u(u,v) = \sum_{x,y} 2w(x,y) [I(x+u, y+v) - I(x,y)] I_x(x+u, y+v)$$

$$E(0,0) = 0$$

$$E_{uu}(u,v) = \sum_{x,y} 2w(x,y) I_x(x+u, y+v) I_x(x+u, y+v)$$

$$E_u(0,0) = 0$$

$$E_v(0,0) = 0$$

$$+ \sum_{x,y} 2w(x,y) [I(x+u, y+v) - I(x,y)] I_{xx}(x+u, y+v)$$

$$E_{uu}(0,0) = \sum_{x,y} 2w(x,y) I_x(x,y) I_x(x,y)$$

$$E_{uv}(u,v) = \sum_{x,y} 2w(x,y) I_y(x+u, y+v) I_x(x+u, y+v)$$

$$E_{vv}(0,0) = \sum_{x,y} 2w(x,y) I_y(x,y) I_y(x,y)$$

$$+ \sum_{x,y} 2w(x,y) [I(x+u, y+v) - I(x,y)] I_{xy}(x+u, y+v)$$

$$E_{uv}(0,0) = \sum_{x,y} 2w(x,y) I_x(x,y) I_y(x,y)$$

Corner Detection by Auto-correlation



- Second-order Taylor expansion of $E(u,v)$ about $(0,0)$:

$$E(u,v) = \sum_{x,y} w(x,y) [I(x+u, y+v) - I(x,y)]^2$$

$$E(u,v) \approx [u \ v] \begin{bmatrix} E_{uu}(0,0) & E_{uv}(0,0) \\ E_{uv}(0,0) & E_{vv}(0,0) \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix} \quad \Rightarrow \quad E(u,v) \approx [u \ v] \underbrace{\begin{bmatrix} \sum_{x,y} w(x,y) I_x^2(x,y) & \sum_{x,y} w(x,y) I_x(x,y) I_y(x,y) \\ \sum_{x,y} w(x,y) I_x(x,y) I_y(x,y) & \sum_{x,y} w(x,y) I_y^2(x,y) \end{bmatrix}}_M \begin{bmatrix} u \\ v \end{bmatrix}$$

$$E(u,v) \approx [u \ v] M \begin{bmatrix} u \\ v \end{bmatrix}$$

$$M = \sum_{x,y} w(x,y) \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix}$$

Corner Detection by Auto-correlation



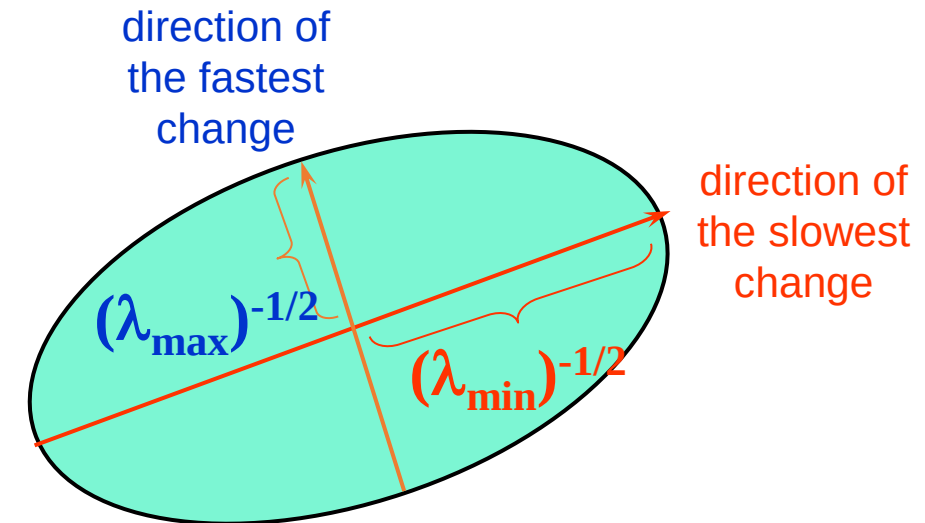
- Interpreting the second moment matrix 二阶矩矩阵
- Consider a horizontal “slice” of $E(u, v)$:
 - This is the equation of an ellipse.

$$M = \sum_{x,y} w(x, y) \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix}$$

$$E(u, v) = [u \ v] M \begin{bmatrix} u \\ v \end{bmatrix} = \text{const}$$

- Diagonalization of M: $M = R^{-1} \begin{bmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \end{bmatrix} R$ 特征值

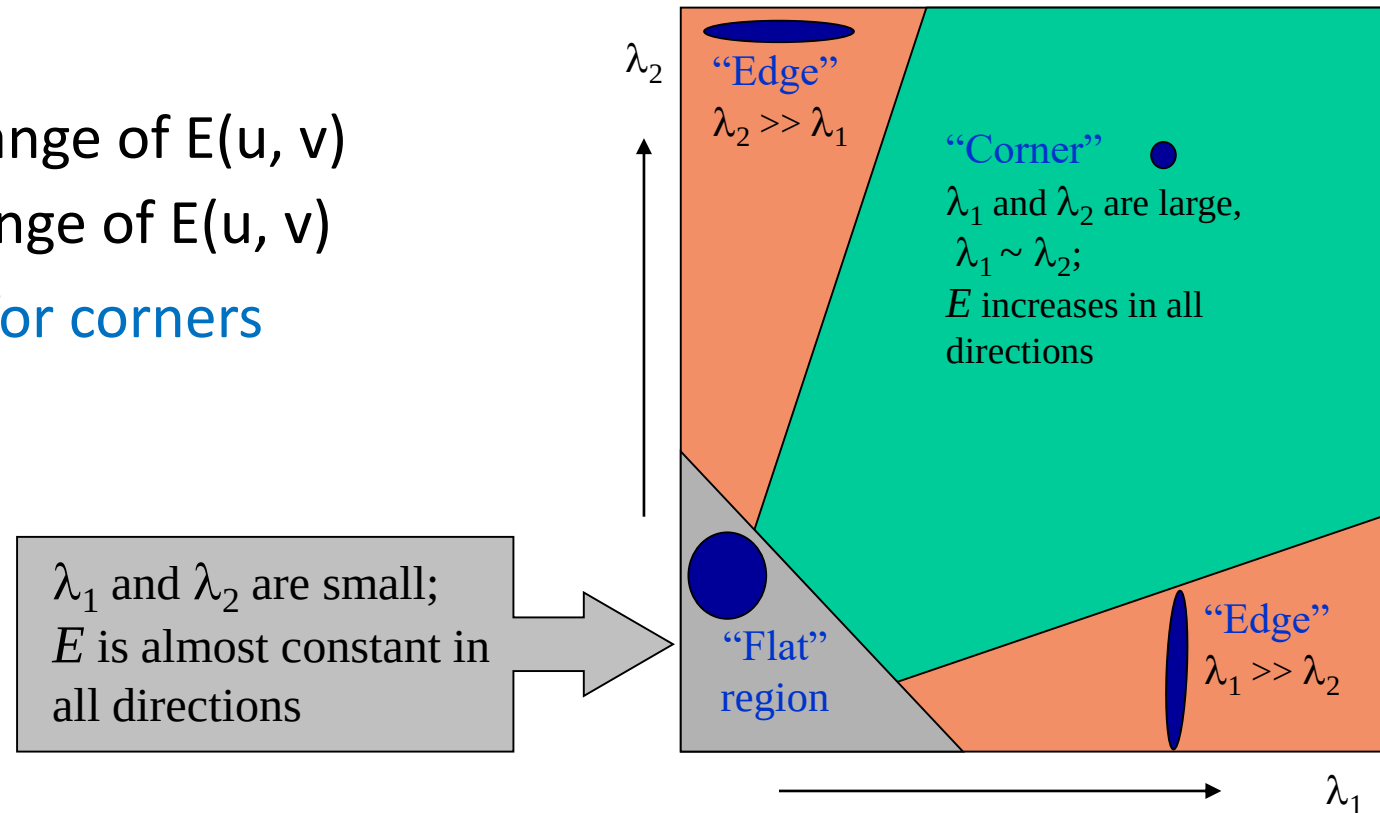
The axis lengths of the ellipse are determined by the eigenvalues,
and the orientation is determined by a rotation matrix R .



Corner Detection by eigenvalues of M

通过矩阵 M 的特征值进行角点检测

- Classification of image points using **eigenvalues of M**
- Large λ indicates rapid change of $E(u, v)$
- Small λ indicates slow change of $E(u, v)$
- Both λ_1 and λ_2 are large for corners



Corner Detection by eigenvalues of M



- Cornerness

$$C = \lambda_1 \lambda_2 - \alpha (\lambda_1 + \lambda_2)^2$$

α : constant (0.04 to 0.06) (Shi and Tmasi, 1994)

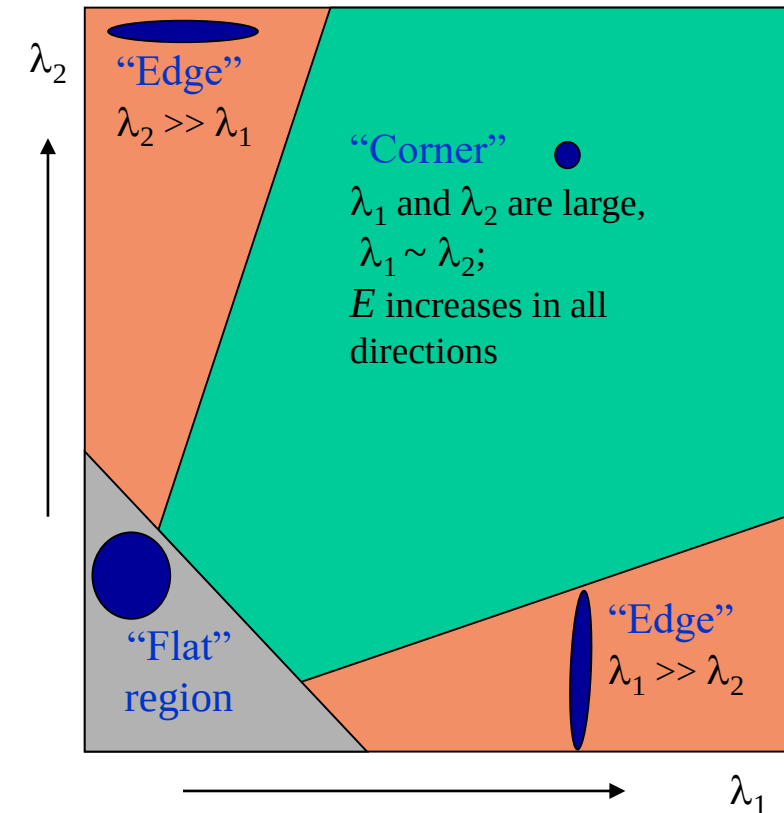
$$C = \det(M) - \alpha \text{trace}(M)^2$$

行列式的值

迹

Determinant: $\det(A) = \prod_{i=1}^n \lambda_i = \lambda_1 \lambda_2 \cdots \lambda_n.$

Trace: $\text{tr}(A) = \sum_i \lambda_i.$



Harris corner detector Harris角点检测



- 角点检测基本步骤

$$M = \sum_{x,y} w(x,y) \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix}$$

$$C = \lambda_1 \lambda_2 - \alpha (\lambda_1 + \lambda_2)^2$$

- 1) Compute ***M matrix*** for each window to recover a ***cornerness score C***.
 - Note: We can find *M* purely from the per-pixel image derivatives!
- 2) Threshold to find pixels which give large corner response (***C > threshold***).
- 3) Find the ***local maxima*** pixels, i.e., suppress non-maxima.

C.Harris and M.Stephens. [“A Combined Corner and Edge Detector.”](#) *Proceedings of the 4th Alvey Vision Conference*: pages 147—151, 1988.

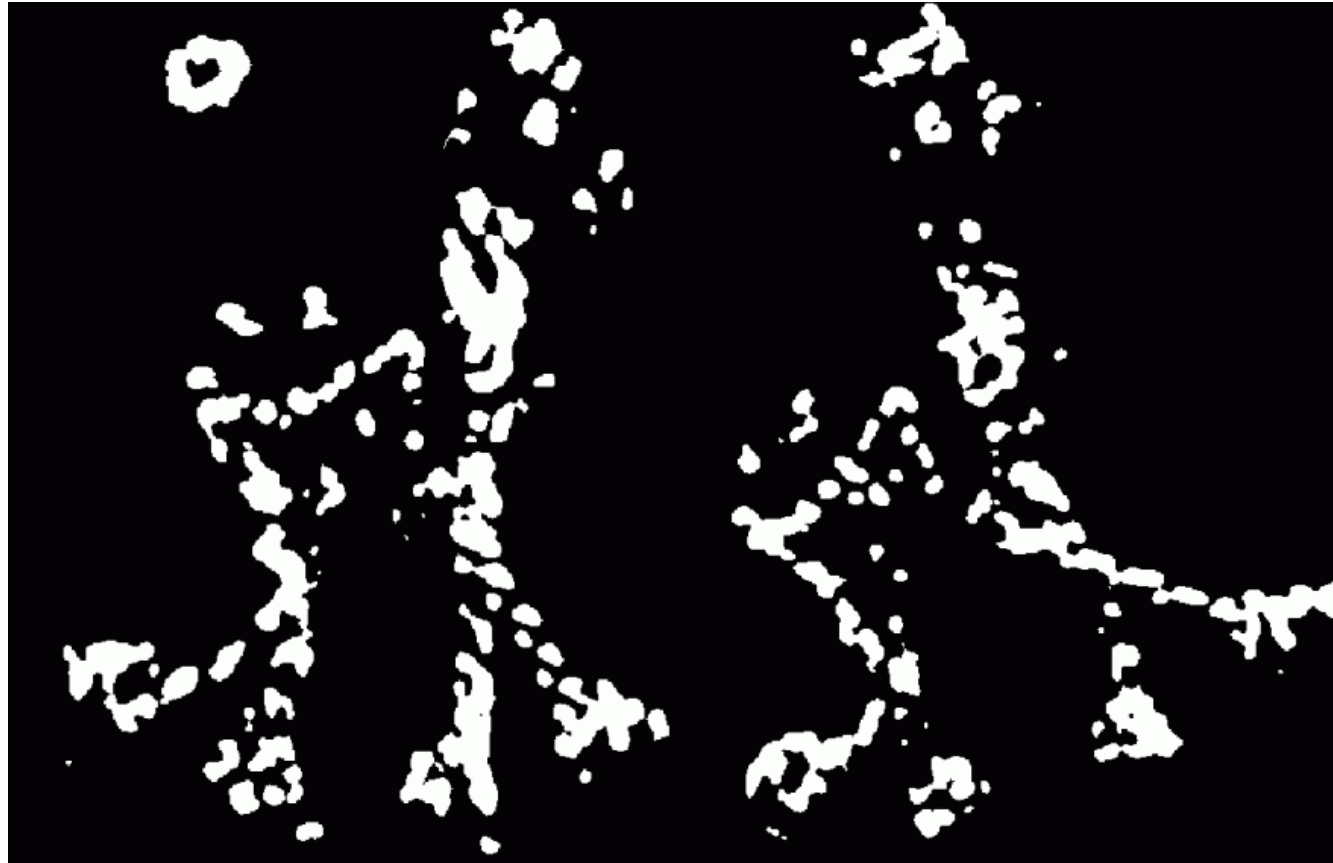
Harris corner detector: example

- Given two images to match



Harris corner detector: example

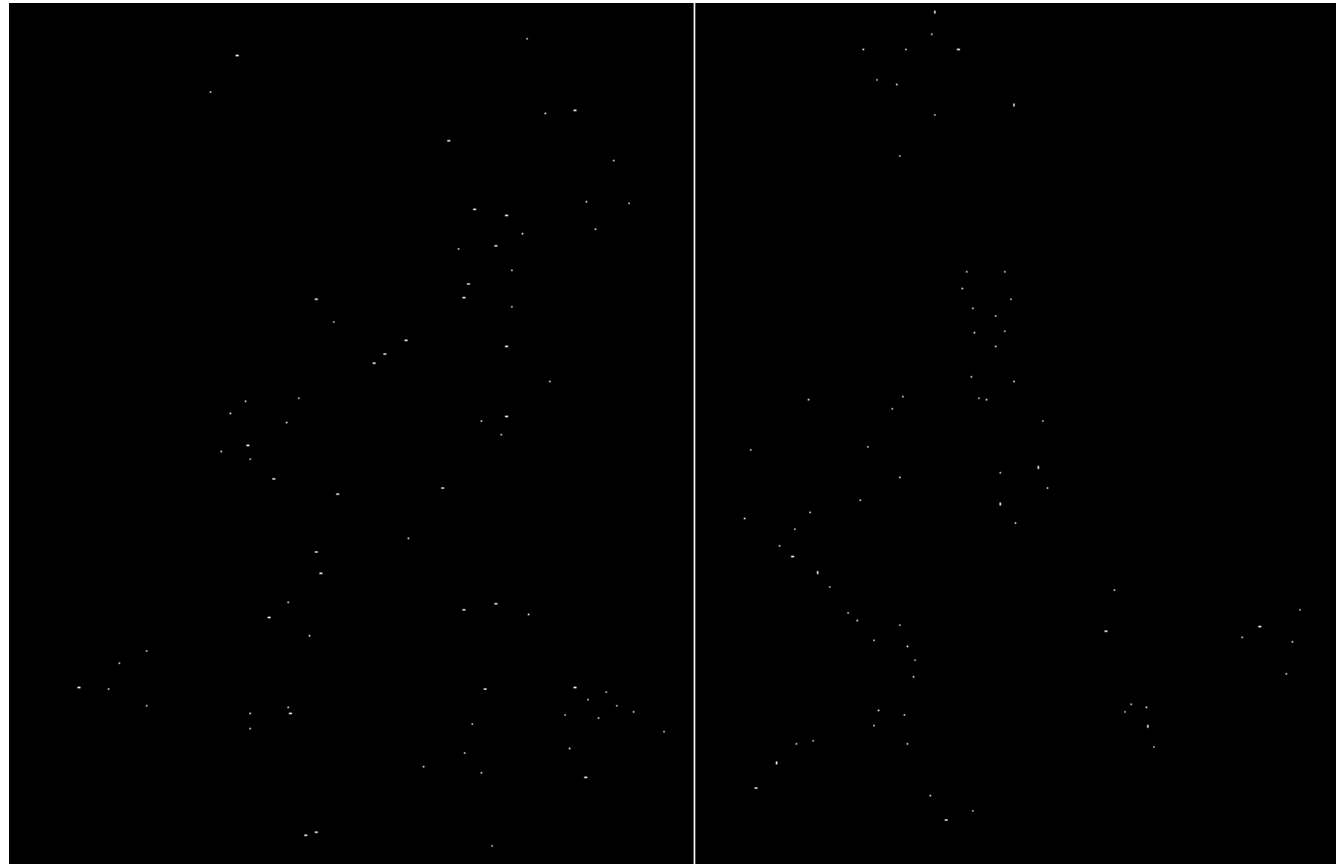
- Find points with large corner response: $C > \text{threshold}$



Harris corner detector: example



- Take only the points of **local maxima of C**



Harris corner detector: example

- Results



Outline



- 1, Corner Detection
- 2, Edge Detection
- 3, Line Detection

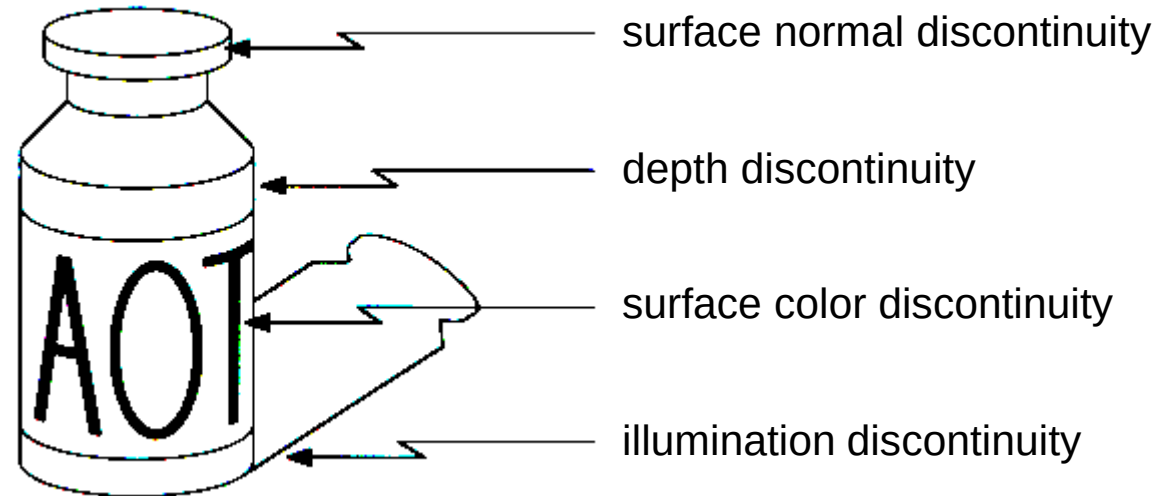
Edge detection 边缘检测

Goal: Identify visual changes (discontinuities) in an image.

Intuitively, semantic information is encoded in edges.

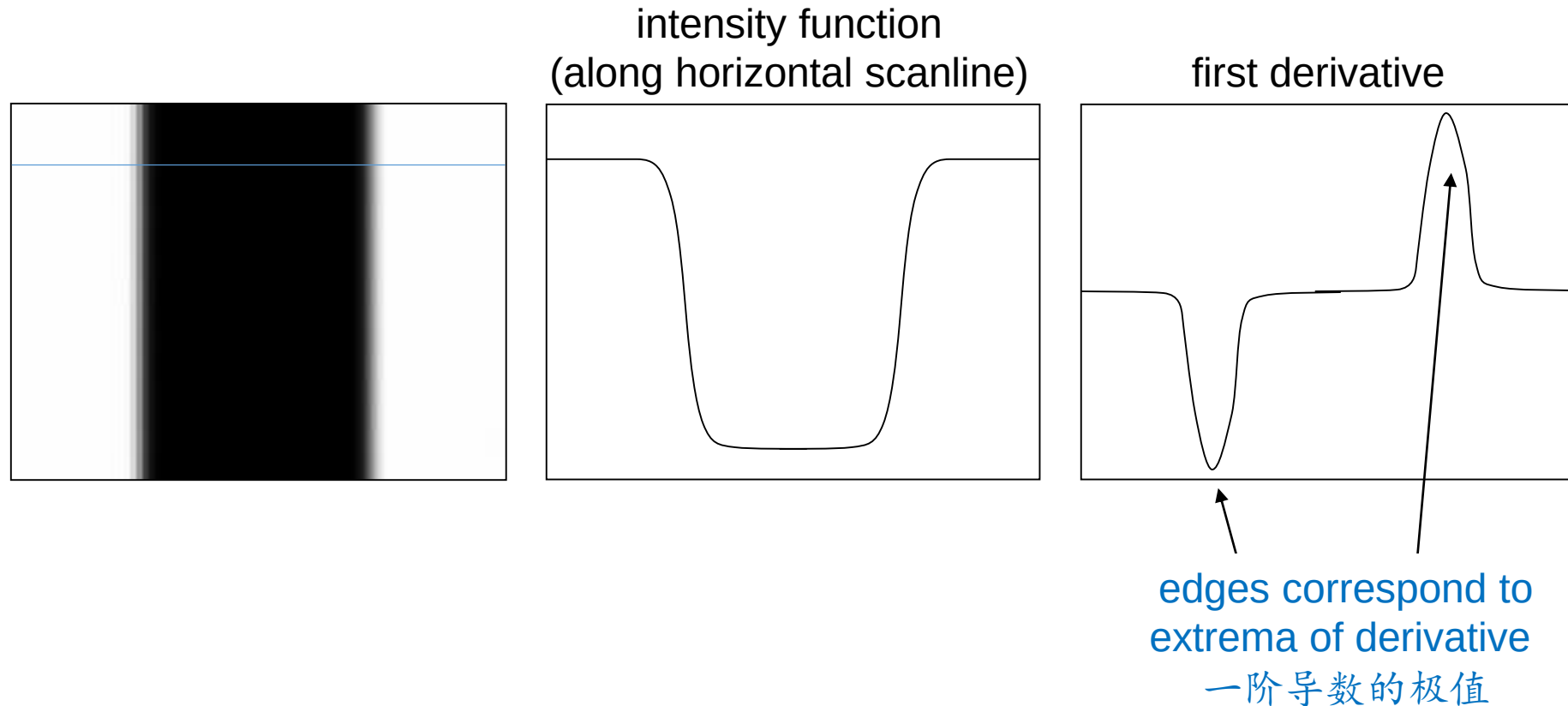


Edges are caused by a variety of factors



Characterizing edges

- An edge is a place of rapid change in the image intensity function



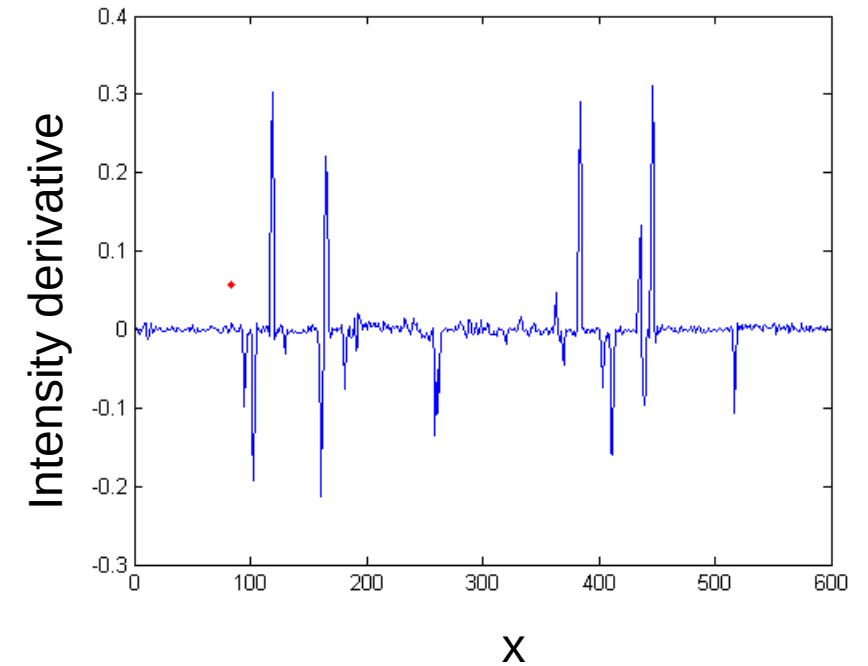
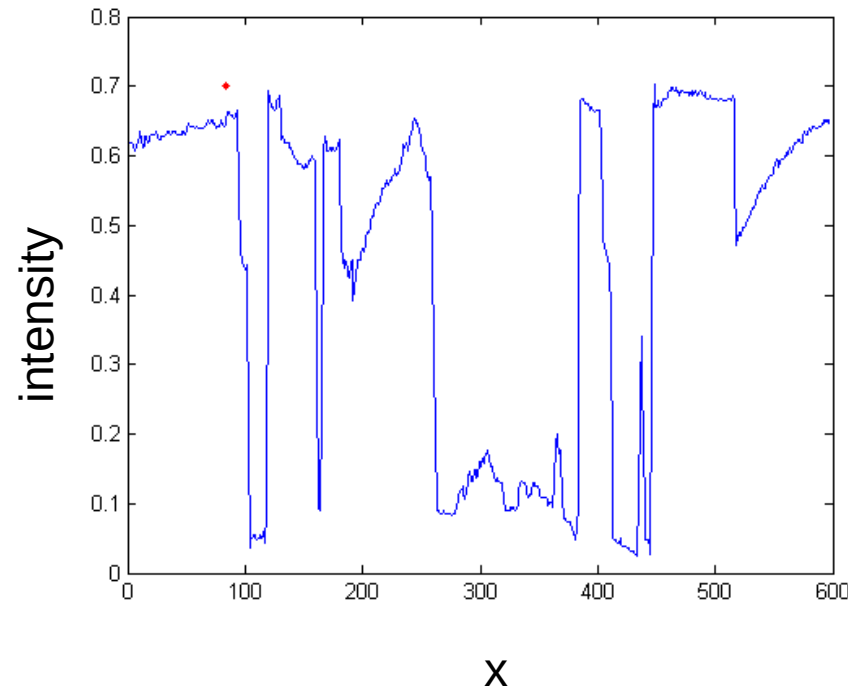
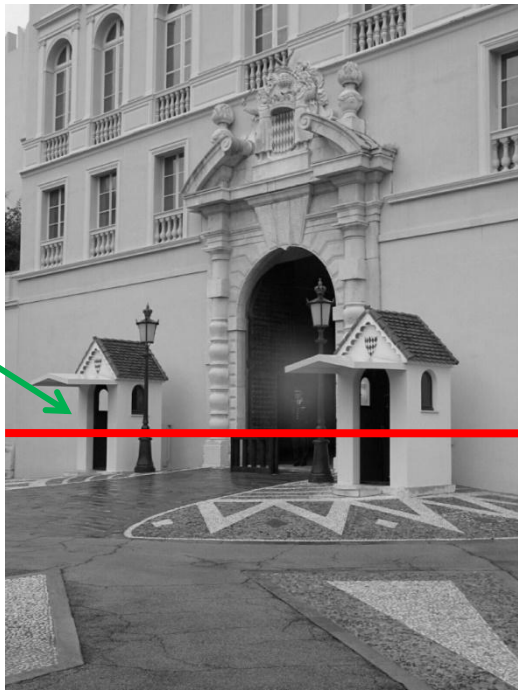
Intensity profile

- An edge is a place of rapid change in the image intensity function
- Corresponds to high frequency in the image

边缘对应图像的高频信息

edges correspond to extrema of derivative

一阶导数的极值

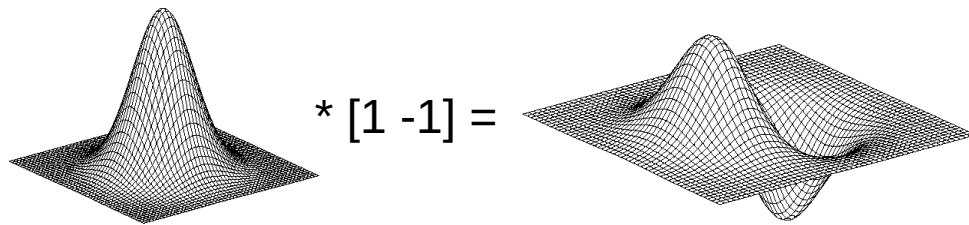


Effects of noise

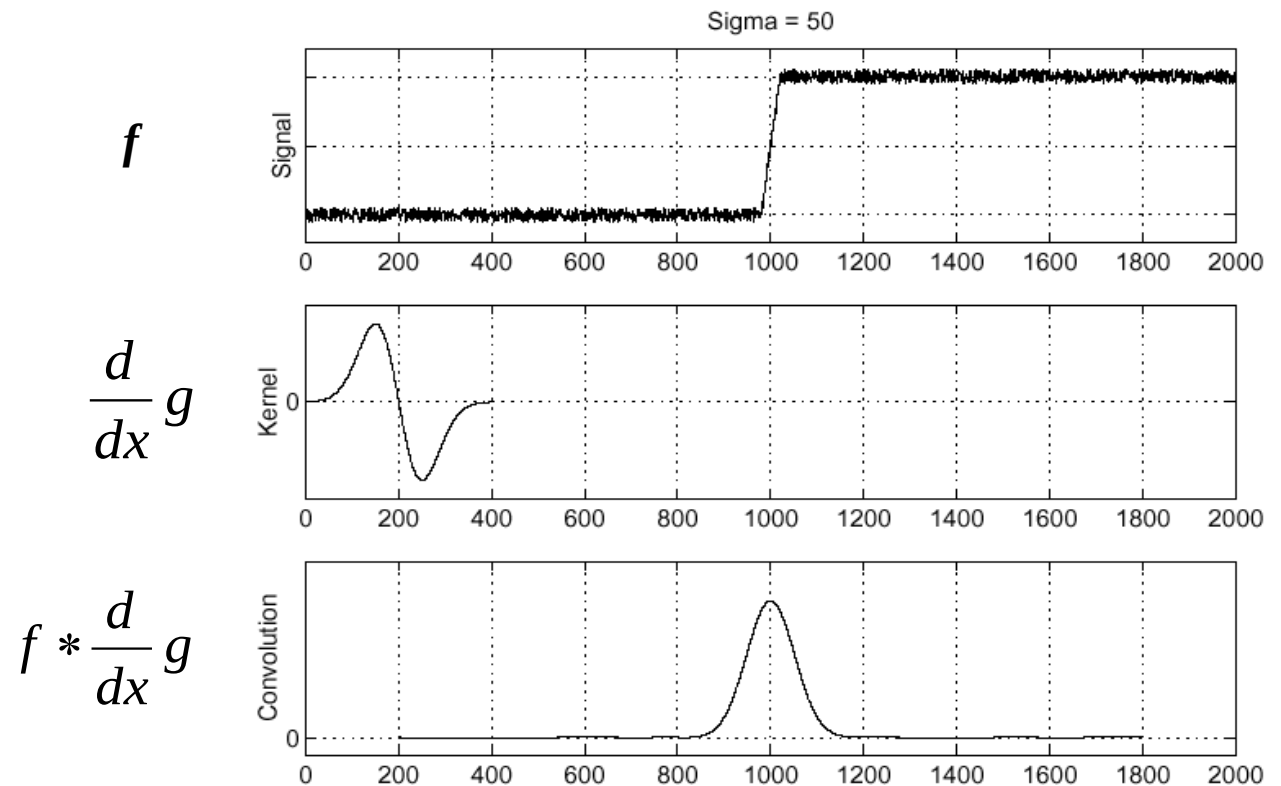
- Derivative theorem of convolution

$$\frac{d}{dx}(f * g) = f * \frac{d}{dx}g$$

- Convolution is differentiable
- This saves us one operation



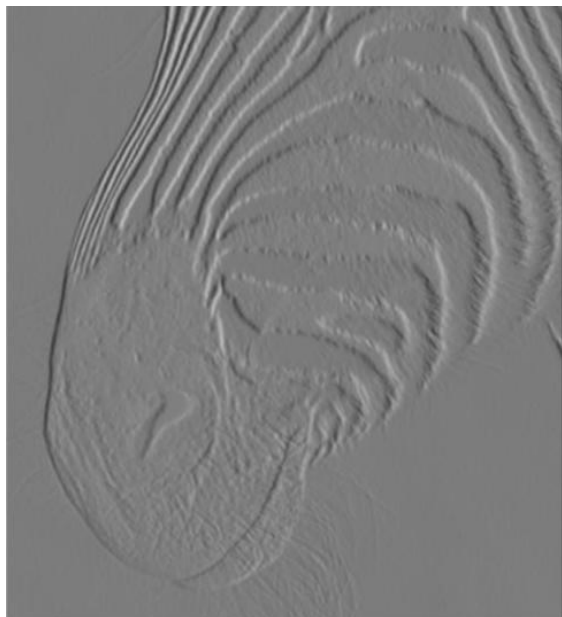
Derivative of 2D Gaussian filter



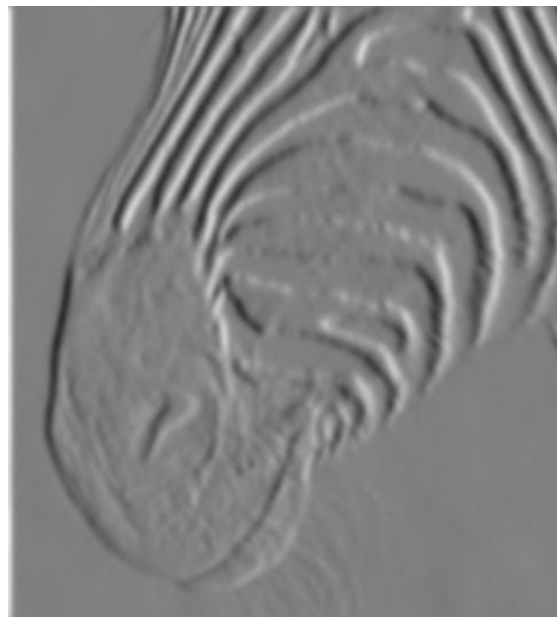
Tradeoff between smoothing and localization

- Smoothed derivative removes noise, but blurs edge.
- Also finds edges at different “scales”.

对图像进行高斯差分滤波的结果



$\sigma = 1$ pixel



$\sigma = 3$ pixels



$\sigma = 7$ pixels

Canny 边缘检测算子



- Probably the most widely used edge detector in computer vision
 - Canny showed that first derivative of Gaussian closely approximates the operator that optimizes the product of *signal-to-noise ratio* and localization
 - Get clean, thin edges that are well connected to nearby edges.



优点:

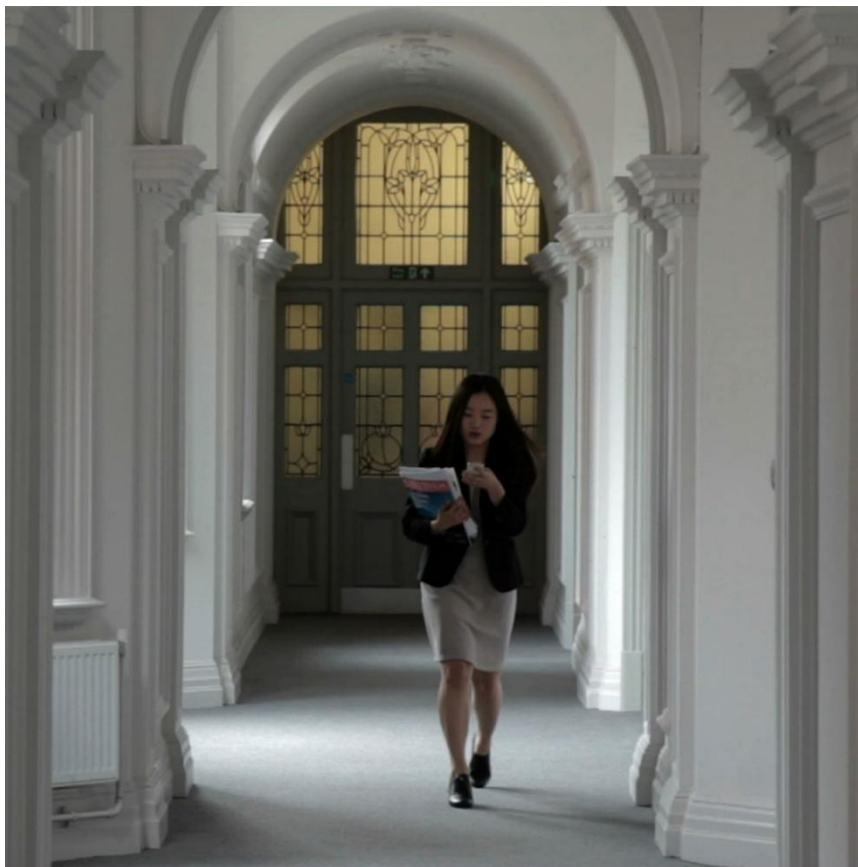
- 高信噪比
- 高定位精度
- 高连续性

J. Canny, [A Computational Approach To Edge Detection](#), IEEE Trans. Pattern Analysis and Machine Intelligence, 8:679-714, 1986. (>35k citations)

Canny 边缘检测算子



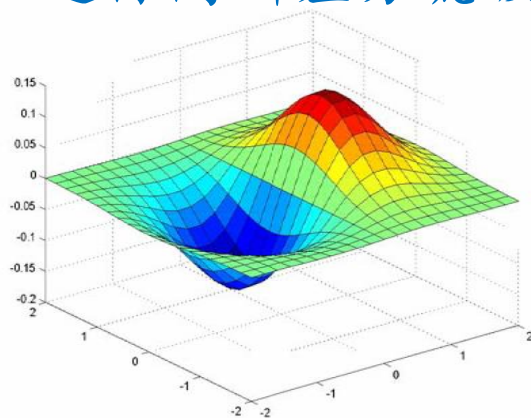
`rgb2gray('img.png')`



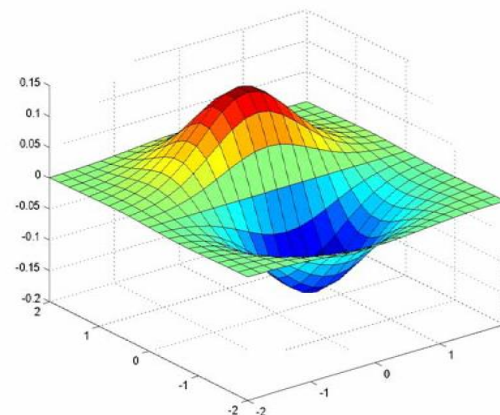
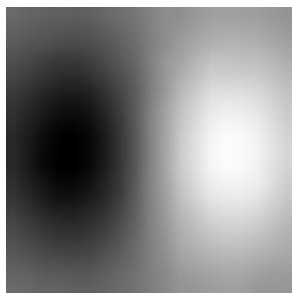
Canny 边缘检测算子

1. Filter image with x, y derivatives of Gaussian

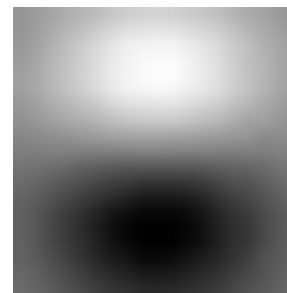
x和y方向上进行高斯差分滤波



x-direction



y-direction



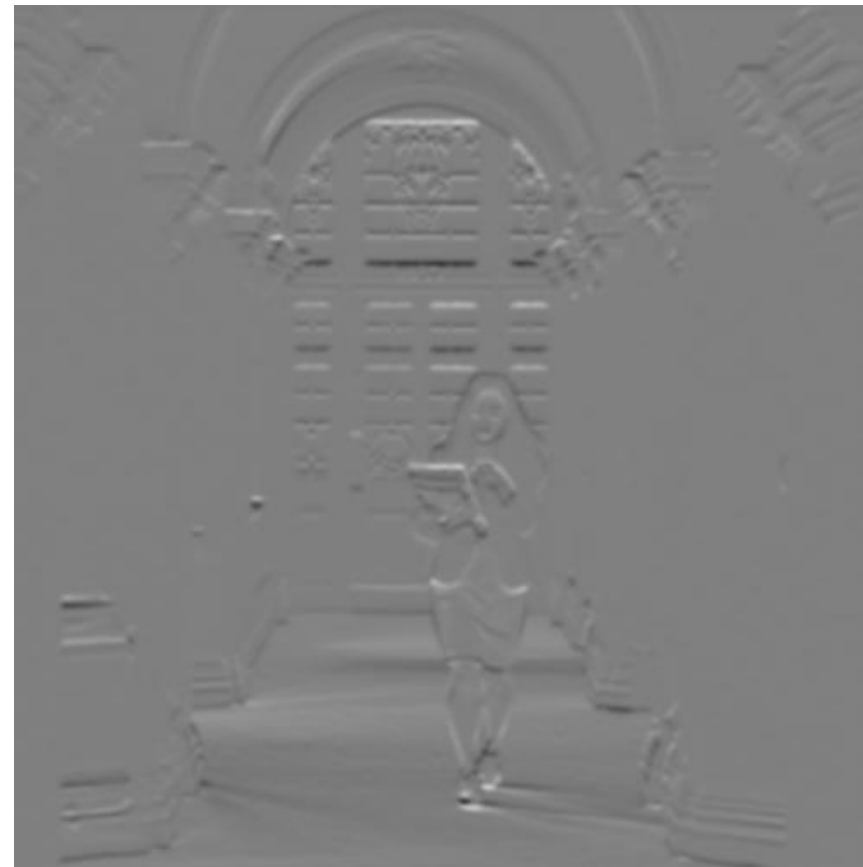
Canny 边缘检测算子



- Computed Gradients



X Derivative of Gaussian G_x



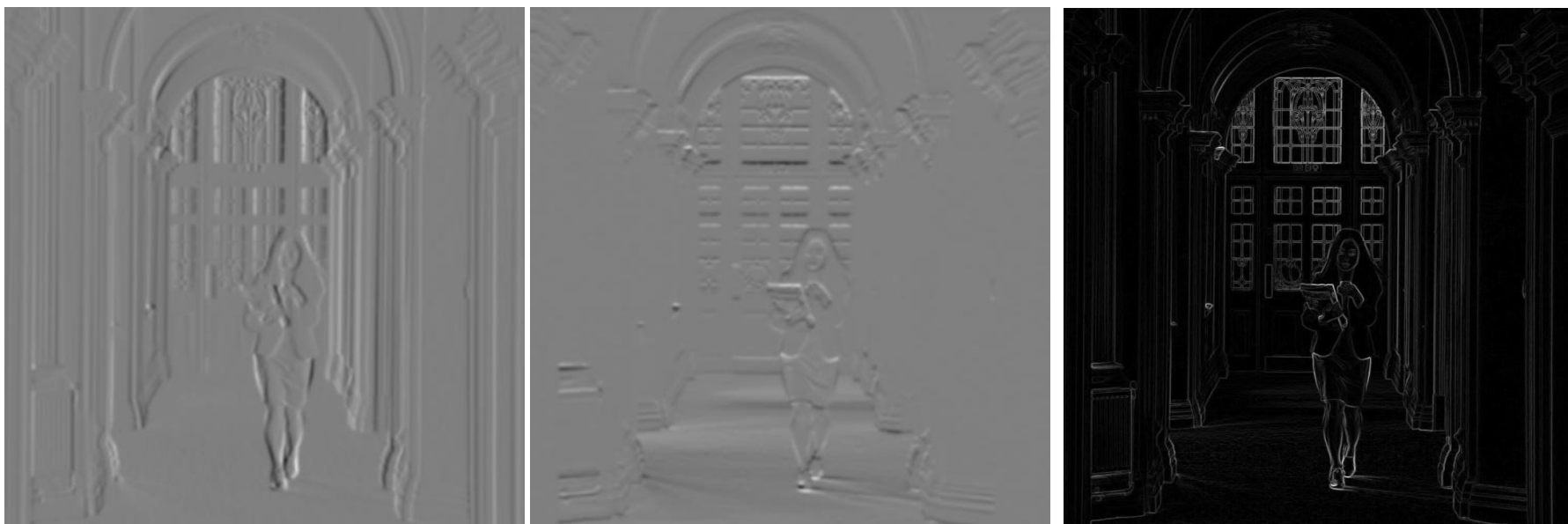
Y Derivative of Gaussian G_y

Canny 边缘检测算子



1. Filter image with x, y derivatives of Gaussian x和y方向上进行高斯差分滤波
2. Find magnitude and orientation of gradient 计算梯度方向和幅值

$$\text{sqrt}(G_x.^2 + G_y.^2) = \text{gradient magnitude}$$



Canny 边缘检测算子



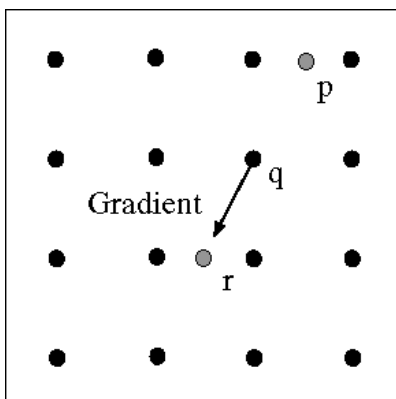
- Threshold magnitude at minimum level
- Get orientation via $\theta = \text{atan2}(G_x, G_y)$



Canny 边缘检测算子

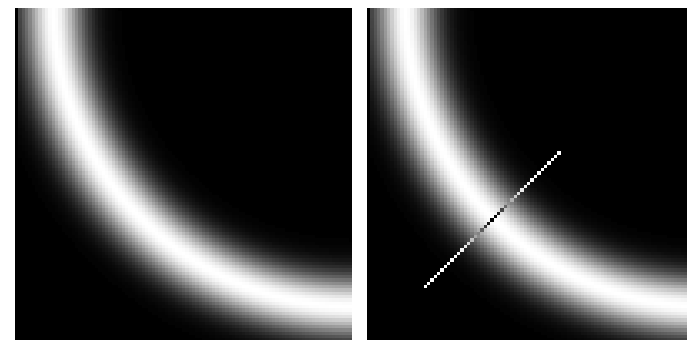


1. Filter image with x, y derivatives of Gaussian x和y方向上进行高斯差分滤波
2. Find magnitude and orientation of gradient 计算梯度方向和幅值
3. Non-maximum suppression: 非最大值抑制
 - Thin multi-pixel wide “ridges” to single pixel width



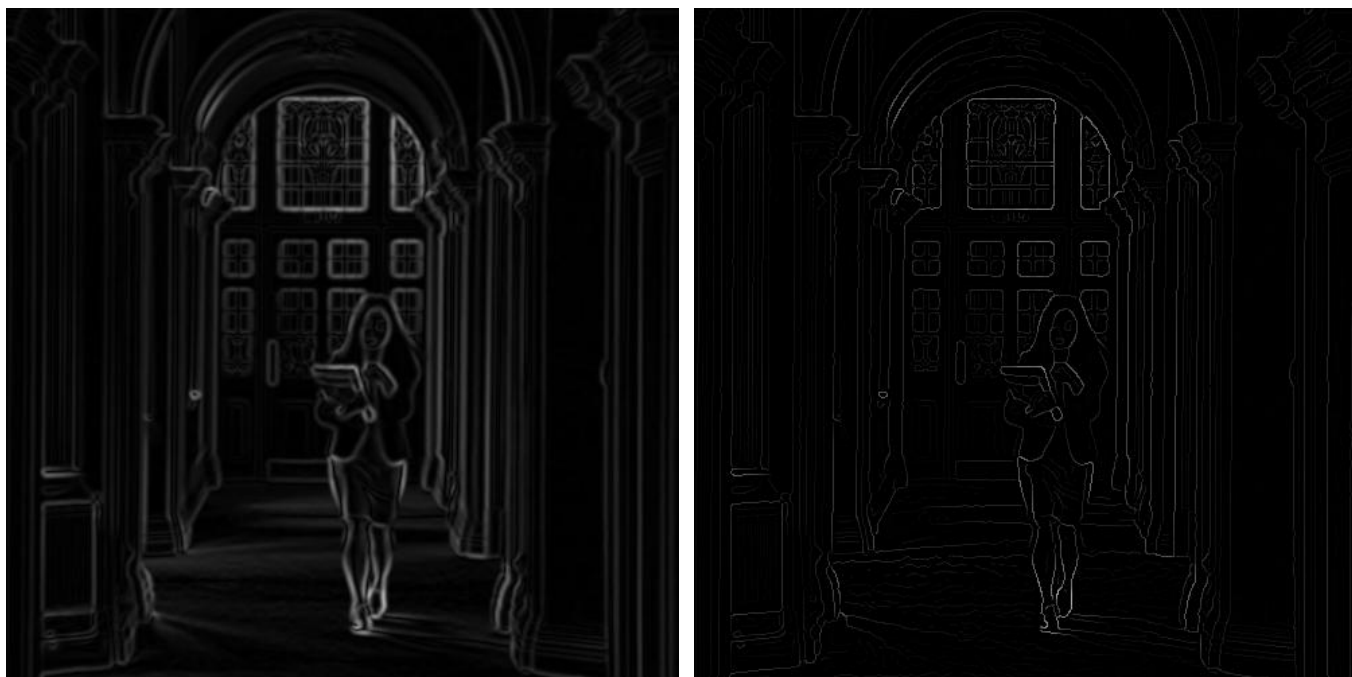
At pixel q:
We have a maximum if the value is larger than those at both p and at r.

Interpolate along gradient direction to get these values.



Canny 边缘检测算子

- Non-max Suppression 非最大值抑制



Gradient magnitude (x4 for visualization)

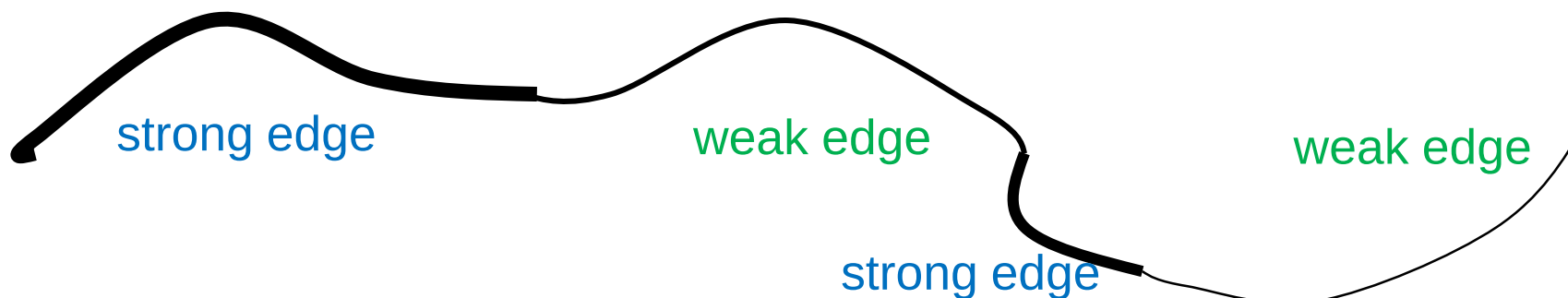
Get 'thin edges'. This might be broken at various points.

Canny 边缘检测算子



1. Filter image with x, y derivatives of Gaussian x和y方向上进行高斯差分滤波
2. Find magnitude and orientation of gradient 计算梯度方向和幅值
3. Non-maximum suppression: 非最大值抑制
 - Thin multi-pixel wide “ridges” to single pixel width
4. ‘Hysteresis’ thresholding and edge linking 边缘连接

滞后阈值法

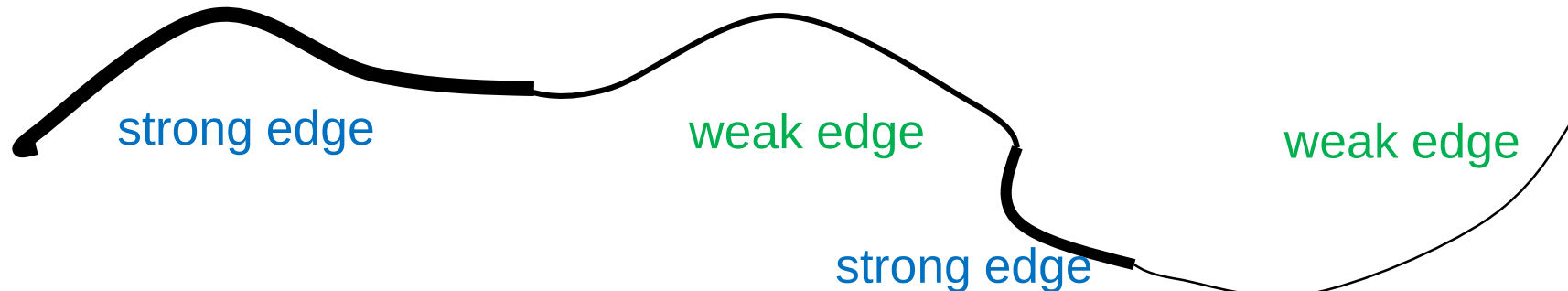


Canny 边缘检测算子



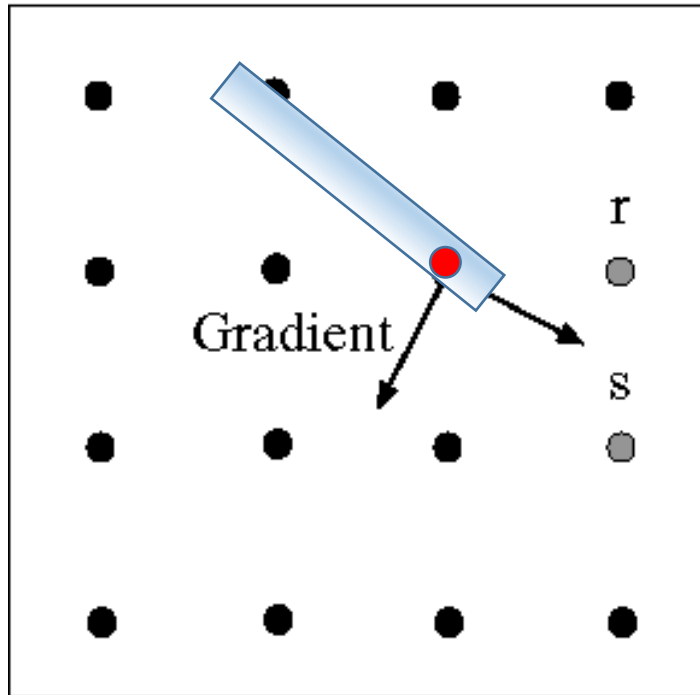
‘Hysteresis’ thresholding 滞后阈值法

- Two thresholds – t_{low} and t_{high}
- Grad. mag. $> t_{high}$? = strong edge
- Grad. mag. $< t_{low}$? noise
- In between = weak edge
- ‘Follow’ edges starting from strong edge pixels
- Continue them into weak edges



Canny 边缘检测算子

Edge linking 边缘连接

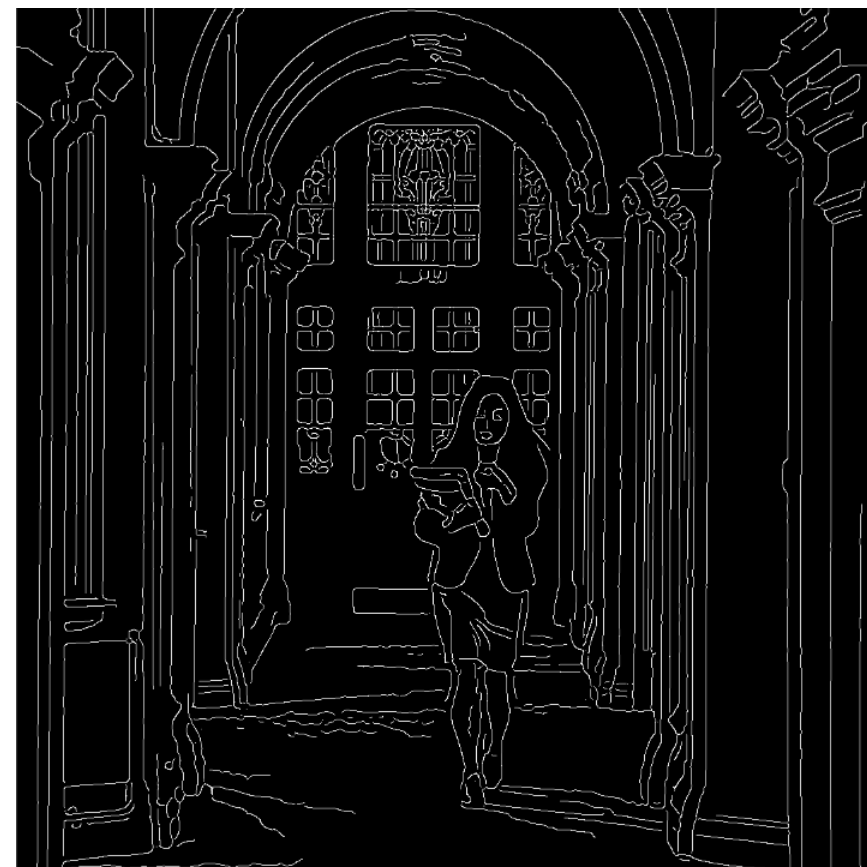


Assume the marked point is an edge point.
Then we construct the tangent to the edge curve
(which is normal to the gradient at that point) and use
this to predict the next points (here either r or s).

Canny 边缘检测算子

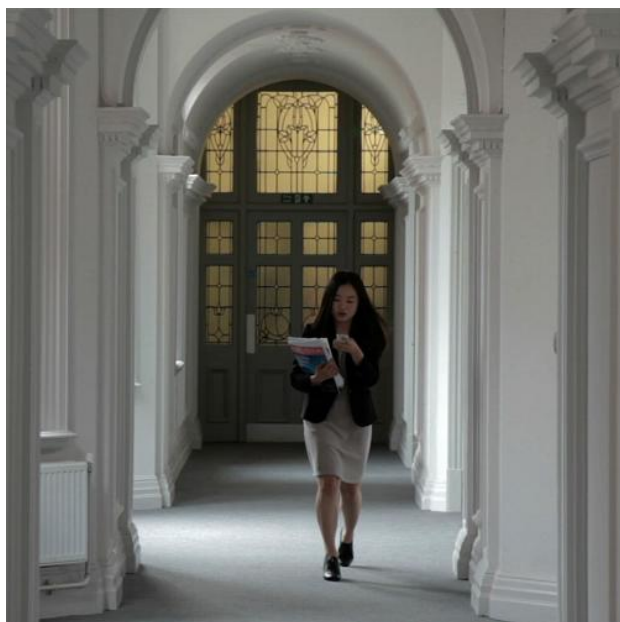
Result

$$\sigma = \sqrt{2}, t_{low} = 0.05, t_{high} = 0.1$$

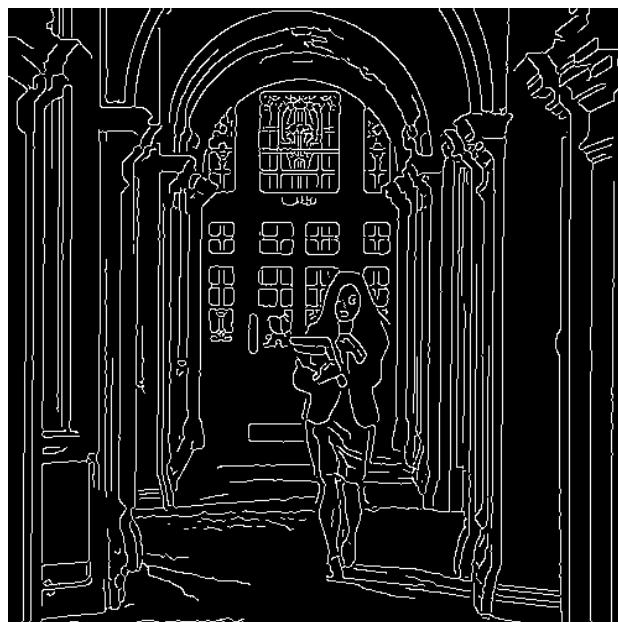


Canny 边缘检测算子

Effect of σ (Gaussian kernel spread/size)

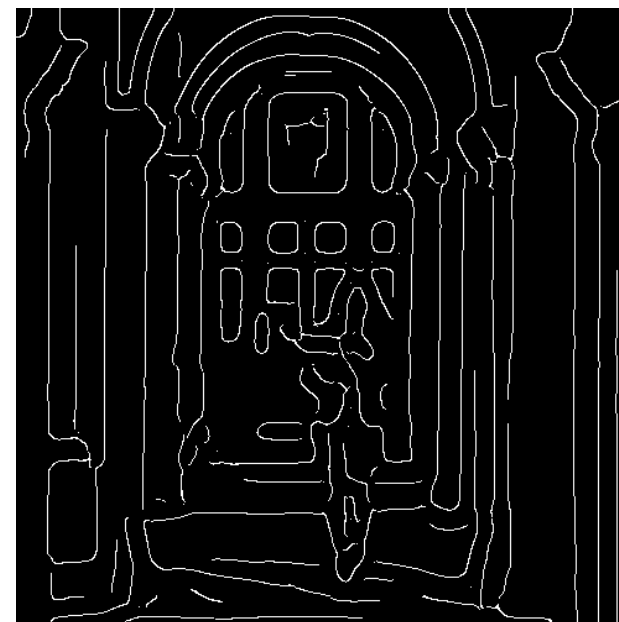


Original



$\sigma = \sqrt{2}$

get fine features



$\sigma = 4\sqrt{2}$

get large scale edges

Canny 边缘检测算子——小结



1. Filter image with x, y derivatives of Gaussian x和y方向上进行高斯差分滤波
 2. Find magnitude and orientation of gradient 计算梯度方向和幅值
 3. Non-maximum suppression: 非最大值抑制
 - Thin multi-pixel wide “ridges” to single pixel width
 4. ‘Hysteresis’ Thresholding: 滞后阈值与边缘连接
 - Define two thresholds: low and high
 - Use the high threshold to start edge curves and the low threshold to continue them
 - ‘Follow’ edges starting from strong edge pixels
 - Connected components (Szeliski 3.3.4)
- Python: e.g., `skimage.feature.canny()`

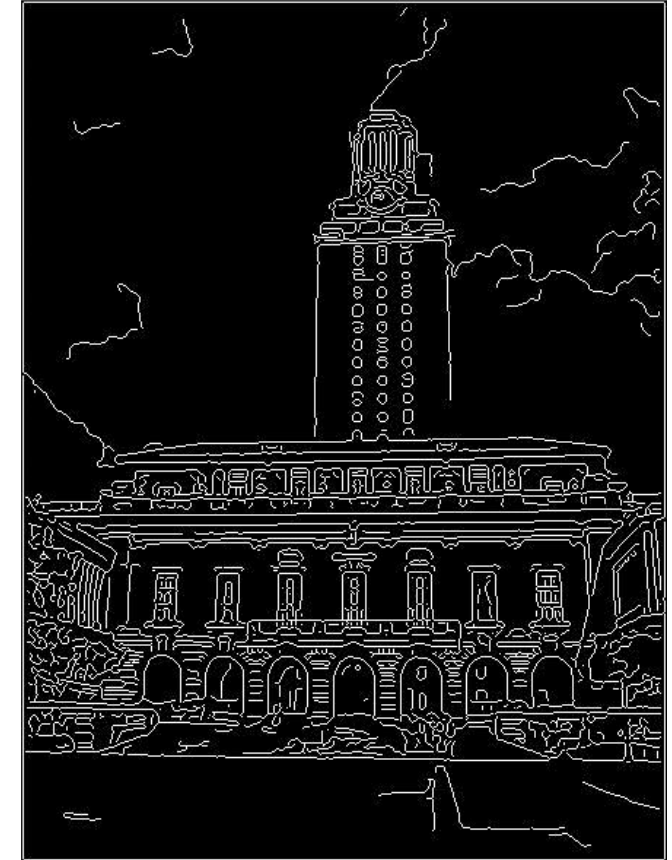
Outline



- 1, Corner Detection
- 2, Edge Detection
- 3, Line Detection

Difficulty of line fitting

- Extra edge points (clutter), **multiple models**:
 - –which points go with which line, if any?
- Only some parts of each line detected, and some parts are **missing**:
 - –how to find a line that bridges missing evidence?
- **Noise** in measured edge points, orientations:
 - –how to detect true underlying parameters?
- It's not feasible to check all combinations of features by fitting a model to each possible subset.



Line detection by voting

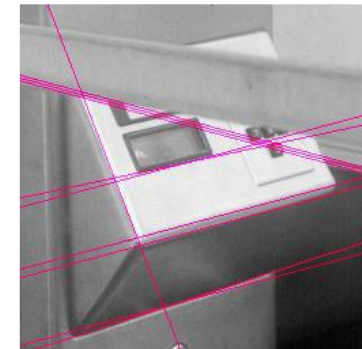
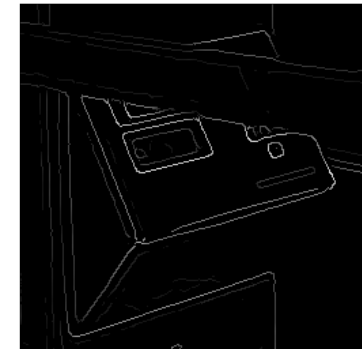
- Questions to answer

- Given points that belong to a line, what is the line?
- How many lines are there?
- Which points belong to which lines?

霍夫变换

- Hough Transform is a voting technique to answer all of these questions.

- 1, Record vote for each possible line on which each edge point lies.
- 2, Look for lines that get many votes.



Finding lines in an image: Hough space

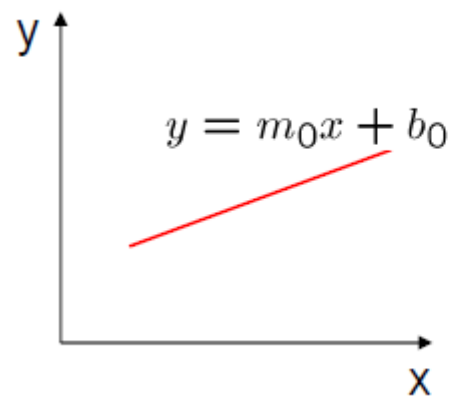
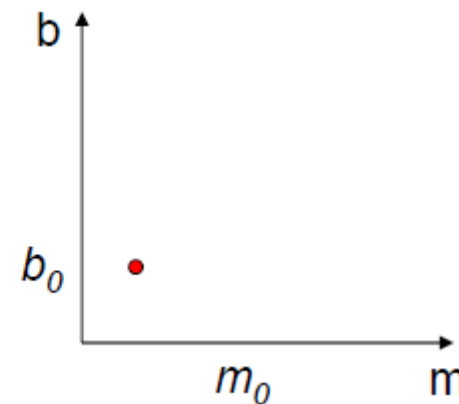


Image space



Hough (parameter) space

- A **line** in the image corresponds to a **point** in Hough space
 - To go from image space to Hough space
 - given a set of points (x,y) , find all (m,b) such that $y = mx + b$

图像空间中一条**直线**，对应霍夫空间中的一个**点**

Finding lines in an image: Hough space

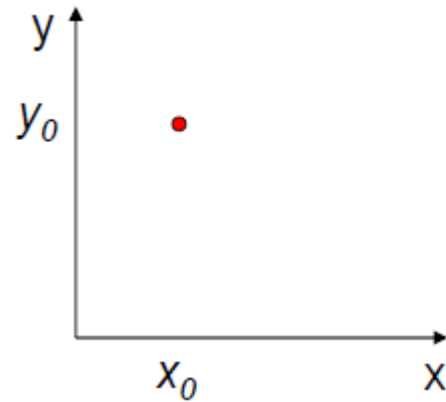
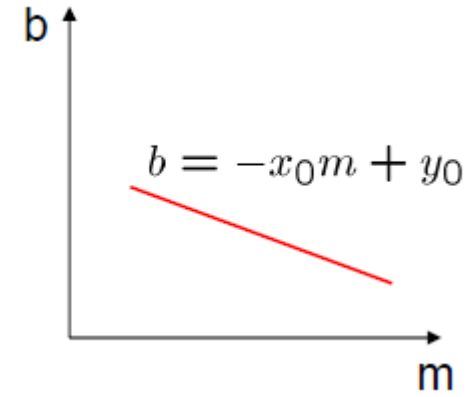


Image space



Hough (parameter) space

- A **point** in the image corresponds to a **line** in Hough space
- What does a point (x_0, y_0) in the image space map to?
 - Answer: the solutions of $b = -x_0 m + y_0$

图像空间中的一个**点**，对应霍夫空间中的一条**直线**

Finding lines in an image: Hough space

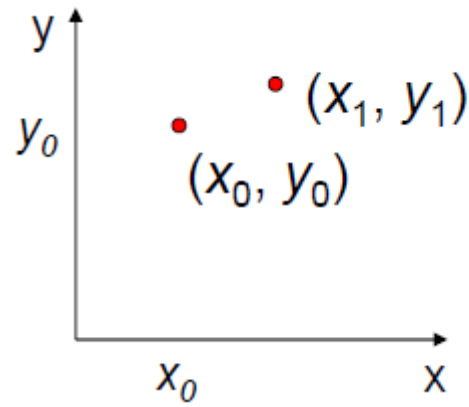
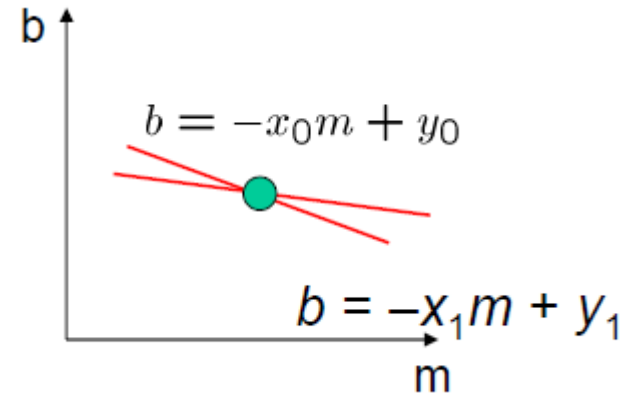


Image space



Hough (parameter) space

- What are the parameters for the line that contains both (x_0, y_0) and (x_1, y_1) ?
 - It is the intersection of the lines $b = -x_0m + y_0$ and $b = -x_1m + y_1$

Finding lines in an image: Hough space

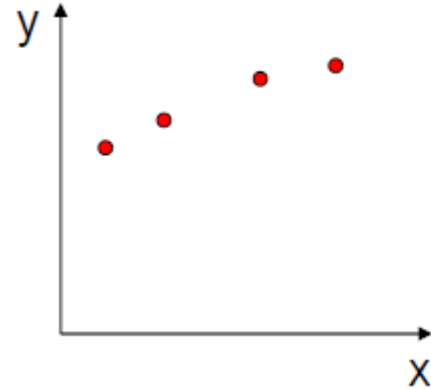
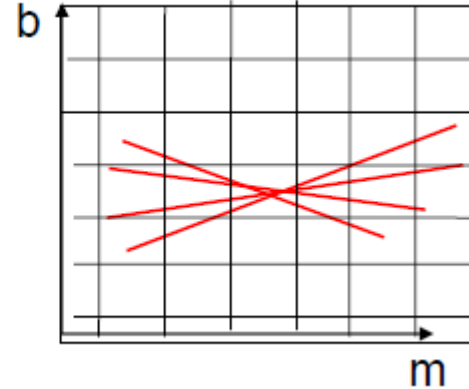


Image space



Hough (parameter) space

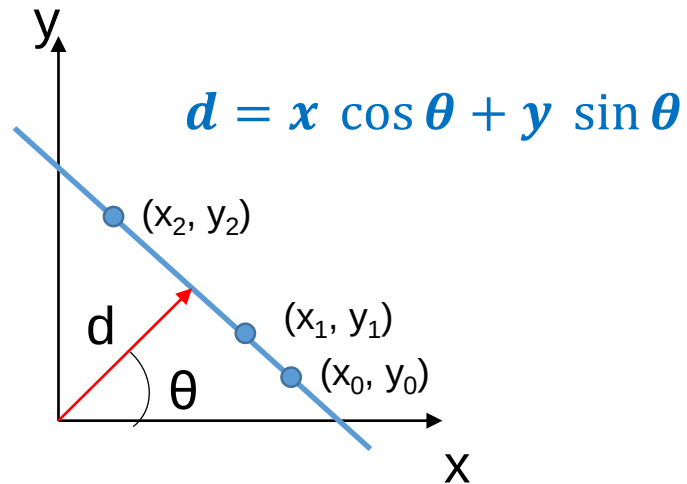
离散化参数
空间

- How to find the (m,b) for the most prominent line in the image?
 - Let each edge point in image space **vote** for a set of possible parameters in Hough space
 - **Accumulate votes** in discrete set of bins; parameters with the most votes indicate line in image space.

Polar representation for lines 极坐标表示



- Issues with usual (m,b) parameter space
 - can take on infinite values
 - undefined for vertical lines
- The polar representation can avoid these issues



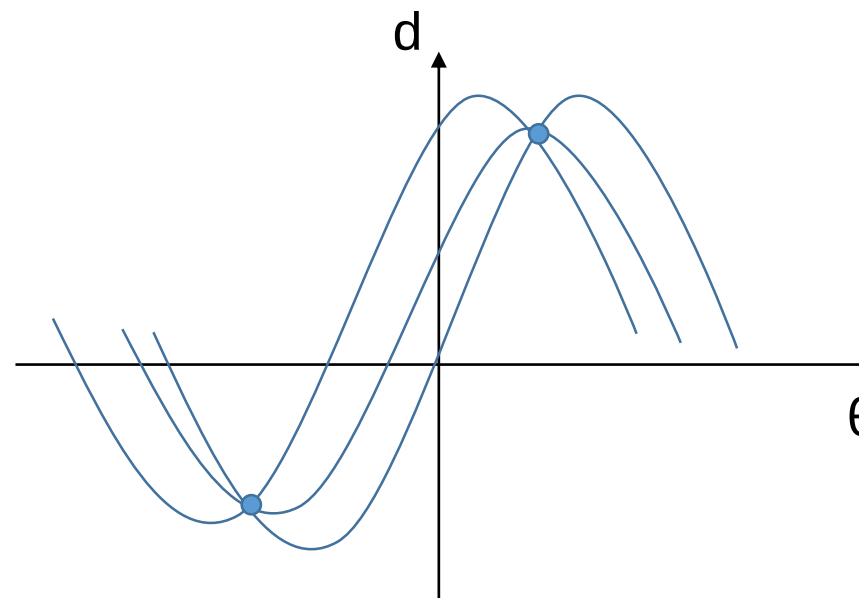
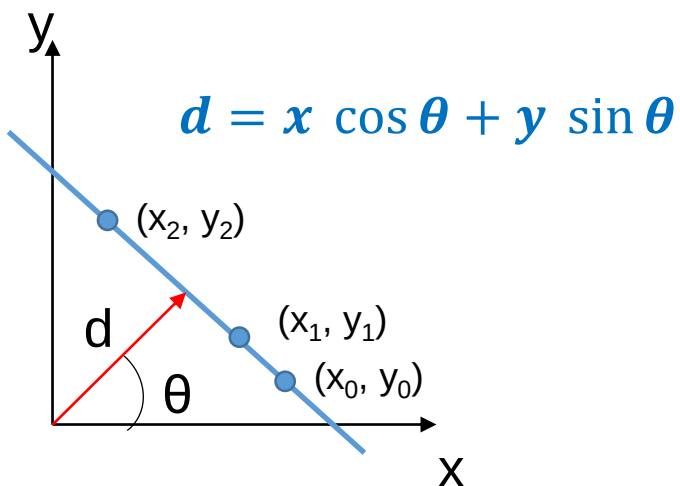
d : perpendicular distance from line to origin

θ : angle the perpendicular makes with the x-axis

Polar representation for lines

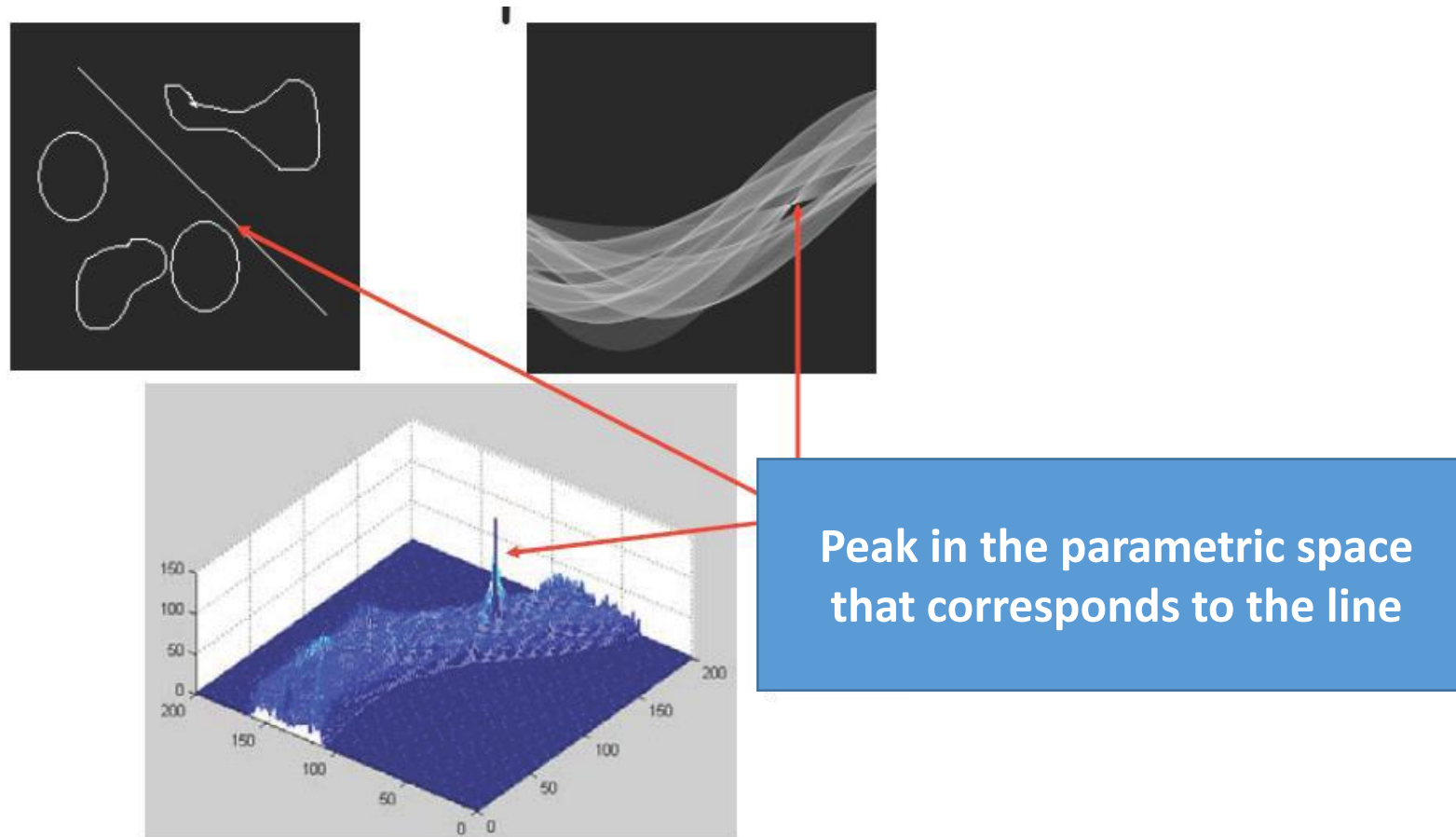
- Point in image space $\rightarrow$ sinusoid segment in Hough space

图像空间中的一个点，对应霍夫空间中的一条正弦曲线



Polar representation for lines

Image space and Hough space



Hough transform algorithm

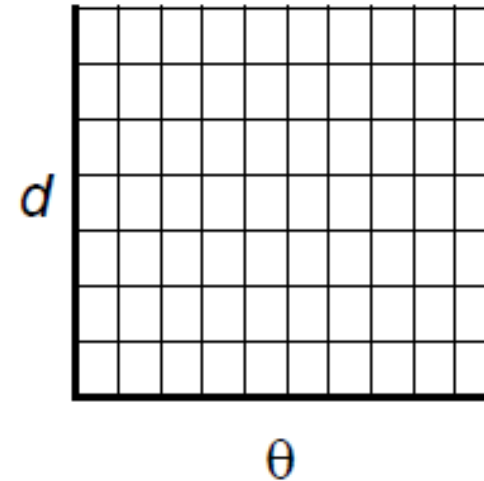


- Using the polar parameterization:

$$d = x \cos \theta + y \sin \theta$$

1. Initialize $H[d, \theta] = 0$
2. For each edge point $I[x, y]$:
For $\theta = \theta_{min}$ to θ_{max} :
 $d = x \cos \theta + y \sin \theta$
 $H[d, \theta] += 1$
3. Find the (d, θ) where $H[d, \theta]$ is maximum

H: accumulator array (votes)



General Hough transform 广义霍夫变换

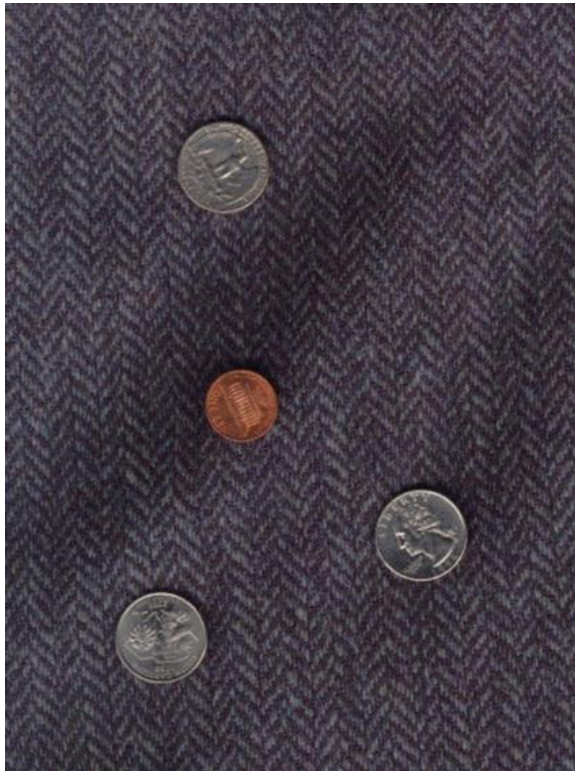


- Parameters for analytic curves

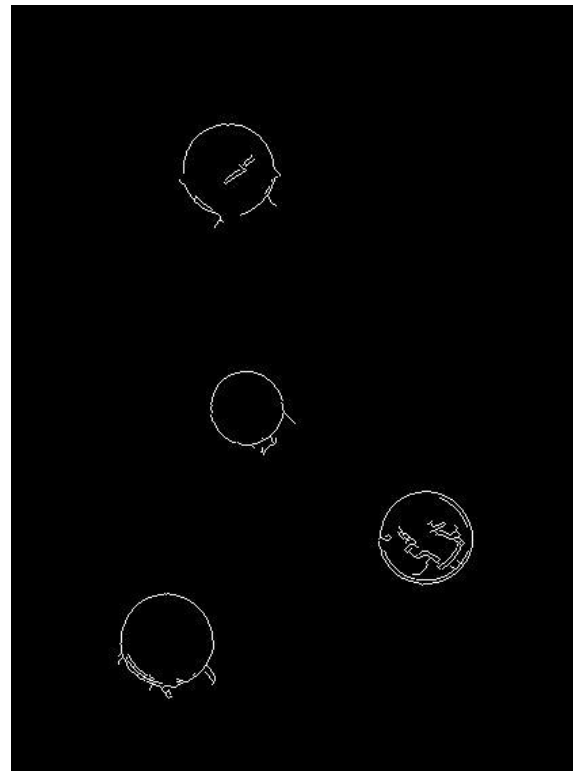
Analytic form	Parameters	Equation
Line	d, θ	$d = x \cos \theta + y \sin \theta$
Circle	x_0, y_0, r	$(x - x_0)^2 + (y - y_0)^2 = r^2$
Parabola	x_0, y_0, r, θ	$(y - y_0)^2 = 4r(x - x_0)$
Ellipse	x_0, y_0, a, b, θ	$\frac{(x - x_0)^2}{a^2} + \frac{(y - y_0)^2}{b^2} = 1$

General Hough transform

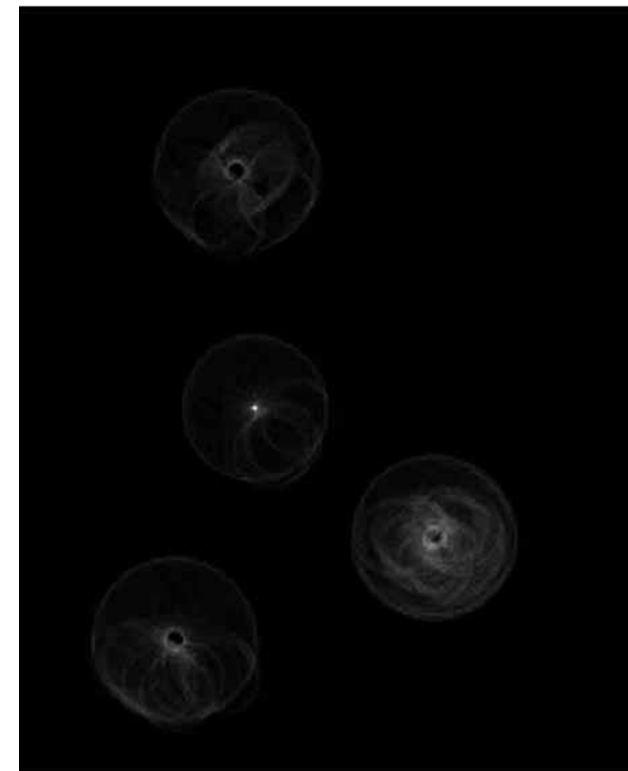
- Example: detecting circles with Hough



Original



Edges



Penny

小结：特征检测

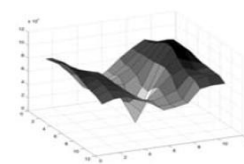
$$E(u, v) = \sum_{x, y} w(x, y) [I(x+u, y+v) - I(x, y)]^2$$

$$E(u, v) \approx [u \ v] M \begin{bmatrix} u \\ v \end{bmatrix}$$

- 1, Harris角点检测
- 2, Canny边缘检测
- 3, 直线检测与霍夫变换

$$M = \sum_{x, y} w(x, y) \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix}$$

$$C = \lambda_1 \lambda_2 - \alpha (\lambda_1 + \lambda_2)^2$$



$\approx$

